

Documentazione Frontend React

Comunicazione con il Backend e Utilizzo dei DTO

Progetto: **SIW Tornei di Calcio**

Università Roma Tre — Sistemi Informativi su Web

1. Introduzione

Questo documento descrive l'architettura del frontend del sistema **Tornei di Calcio**, realizzato con **React** e **TypeScript**, e illustra come esso comunica con il backend **Spring Boot**. Verranno inoltre analizzati i **DTO** (Data Transfer Object) utilizzati per il trasferimento dei dati tra backend e frontend, spiegandone lo scopo e il modo in cui vengono impiegati su entrambi i lati dell'applicazione.

2. Architettura Generale

L'applicazione segue un'architettura **client-server** moderna:

- **Frontend**: Single Page Application (SPA) scritta in React 18, transpilata con Vite.
- **Backend**: Applicazione Spring Boot 4.0.6 con Java 17, Spring MVC, Spring Data JPA, Spring Security e Thymeleaf.
- **Database**: PostgreSQL per la persistenza dei dati.
- **Comunicazione**: REST API JSON tramite HTTP; autenticazione session-based con cookie JSESSIONID.

In produzione, Spring Boot serve la build statica di React (cartella **src/main/resources/static/app**), quindi frontend e backend condividono lo stesso dominio e non sono necessarie configurazioni CORS complesse. In sviluppo, invece, Vite gira su **localhost:5173** mentre Spring Boot è su **localhost:8080**; per questo il frontend utilizza una variabile d'ambiente **VITE_API_BASE_URL** per puntare al backend.

3. Il Frontend React

3.1 Tecnologie e Dipendenze

Il frontend si trova nella cartella **frontend/** ed è un progetto Node.js gestito con npm. Le dipendenze principali sono:

- **react** ^18.3.1 e **react-dom** ^18.3.1: librerie core per la costruzione dell'interfaccia utente.
- **react-router-dom** ^6.28.0: gestione del routing lato client.
- **axios** ^1.18.0: client HTTP per le chiamate API al backend.
- **framer-motion** ^12.40.0: libreria per animazioni (opzionale, attualmente inclusa).
- **tailwindcss** ^4.3.1: framework CSS utility-first per lo stile.
- **vite** ^5.4.11: build tool per il bundling e il development server.

3.2 Struttura del Progetto

La cartella **src/** del frontend è organizzata in sottocartelle logiche:

- **components/**: componenti riutilizzabili (Navbar, Hero, TorneoCard, FadeIn, TeamCarousel, etc.).
- **pages/**: componenti pagina (HomePage).
- **context/**: React Context per lo stato globale (AuthContext).
- **services/**: funzioni di comunicazione con il backend (api.ts, homeService.ts, torneoService.ts).
- **types/**: definizioni TypeScript dei tipi e delle interfacce (equivalenti ai DTO del backend).
- **styles/** e ***.css**: fogli di stile custom.

3.3 Componenti Principali

3.3.1 App.tsx

È il componente radice. Avvolge l'applicazione in **BrowserRouter** (fornito da react-router-dom) e in **AuthProvider** (contesto di autenticazione). Include la Navbar e le rotte definite in **AppRoutes**.

```
export default function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Navbar />
        <AppRoutes />
      </AuthProvider>
    </BrowserRouter>
  );
}
```

3.3.2 AppRoutes.tsx

Definisce le rotte dell'applicazione con il componente **Routes** di React Router. La rotta **/** punta alla HomePage. Le rotte protette (come **/admin**) sono avvolte in **PrivateRoute**, che reindirizza al login se l'utente non è autenticato.

```
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route element={<PrivateRoute />>
    <Route path="/admin" element={<div>Admin Page</div>} />
  </Route>
</Routes>
```

```

    </Route>
    <Route path="" element={<div>404 Not Found</div>} />
  </Routes>

```

3.3.3 AuthContext.tsx

Gestisce lo stato globale dell'autenticazione. Effettua una chiamata all'endpoint **/api/auth/me** al mount dell'app per verificare se l'utente ha una sessione valida. Espone: **isAuthenticated**, **isAdmin**, **username**, **loading**, **refresh** e **logout**.

```

const API_BASE = import.meta.env.VITE_API_BASE ?? '';

const refresh = async () => {
  const res = await fetch(`${API_BASE}/api/auth/me`, {
    credentials: 'include', // invia il cookie JSESSIONID
  });
  if (res.ok) {
    const data = await res.json();
    setIsAuthenticated(true);
    setUsername(data.username);
    setIsAdmin(!!data.isAdmin);
  }
};

```

3.3.4 HomePage.tsx

È la pagina principale. Carica i dati dalla home tramite **getHome()** e li visualizza: sezione hero, elenco tornei con ricerca, preferiti, prossime partite e carosello squadre. Implementa anche la logica di toggle dei preferiti (aggiungi/rimuovi) con aggiornamento ottimistico dello stato locale.

3.3.5 TorneoCard.tsx

Componente presentazionale che riceve un oggetto **TorneoHomeDto** e mostra: nome, anno, numero squadre, descrizione e le prossime partite. Mostra il pulsante preferito solo se l'utente è autenticato.

4. Comunicazione con il Backend

4.1 Axios e Configurazione Base

Il file **frontend/src/services/api.ts** crea un'istanza centralizzata di **Axios** con le configurazioni necessarie per la sessione:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || '',
  withCredentials: true, // fa viaggiare il cookie JSESSIONID
  headers: {
    'Content-Type': 'application/json',
    Accept: 'application/json',
  },
});
```

L'opzione **withCredentials: true** è fondamentale: permette al browser di inviare e ricevere i cookie HTTP (in particolare il **JSESSIONID** di Spring Security) nelle richieste cross-origin. In sviluppo, quando il frontend su **:5173** chiama il backend su **:8080**, senza questa opzione il cookie non verrebbe incluso e l'utente risulterebbe sempre non autenticato.

4.2 Autenticazione Session-Based

Il backend utilizza **Spring Security** con autenticazione basata su sessione HTTP. Quando un utente effettua il login tramite il form di Spring Security (**/login**), il server crea una sessione e invia al browser un cookie **JSESSIONID**. Ogni successiva richiesta del frontend deve includere questo cookie per identificare l'utente.

4.3 Protezione CSRF

Spring Security abilita di default la protezione CSRF. Le richieste di modifica (POST, PUT, DELETE) richiedono un token CSRF valido. Il frontend lo ottiene dall'endpoint dedicato **/api/csrf** e lo invia nell'header corrispondente. Il logout, ad esempio, funziona così:

```
const csrfRes = await api.get('/api/csrf');
const { headerName, token } = csrfRes.data;

await api.post('/logout', null, {
  headers: { [headerName]: token },
});
```

4.4 Endpoint API Utilizzati

Il frontend interagisce con i seguenti endpoint REST esposti dal backend:

Endpoint	Metodo	Scopo
/api/home	GET	Recupera dati aggregati per la home page (tornei, squadre, preferenze)
/api/auth/me	GET	Verifica sessione utente corrente

/api/csrf	GET	Ottieni token CSRF per richieste sicure
/api/tornei/{id}/partite	GET	Elenco partite di un torneo
/api/tornei/{id}/squadre	GET	Elenco squadre iscritte a un torneo
/api/tornei/{id}/preferito	POST	Aggiunge un torneo ai preferiti dell'utente
/api/tornei/{id}/preferito	DELETE	Rimuove un torneo dai preferiti
/api/utente/preferiti	GET	Recupera gli ID dei tornei preferiti dall'utente
/logout	POST	Invalida la sessione e cancella il cookie JSESSIONID

4.5 Esempio di Chiamata: `getHome`

Il servizio **homeService.ts** è un esempio tipico di come il frontend richiede i dati:

```
import api from './api';
import type { HomeDto } from '../types';

export const getHome = () => api.get<HomeDto>('/api/home');
```

Notare l'uso di **api.get<HomeDto>**: TypeScript garantisce che i dati restituiti rispettino la struttura dell'interfaccia **HomeDto**, definita nel file **types/index.ts**. Questo permette autocompletamento, type-checking e refactoring sicuro.

5. I DTO (Data Transfer Object)

5.1 Cosa sono i DTO

I **DTO** (Data Transfer Object) sono oggetti leggeri progettati per trasportare dati tra processi, in questo caso tra il backend Spring Boot e il frontend React. Il loro scopo principale è **separare il modello di dominio** (le entità JPA con relazioni, metodi di business e logica di persistenza) dalla **rappresentazione esterna** dei dati che viene serializzata in JSON e inviata al client.

5.2 Perché usare i DTO

- **Sicurezza:** evitano di esporre campi sensibili dell'entità (es. password, dati interni).
- **Performance:** riducono la dimensione del payload JSON includendo solo i dati necessari.
- **Decoupling:** il frontend non dipende dalla struttura interna del database o delle entità JPA.
- **Stabilità:** cambiamenti nel modello di dominio non impattano direttamente il contratto API.
- **Composizione:** permettono di aggregare dati da più entità in un'unica struttura ottimizzata per la UI.

5.3 I DTO del Backend (Java Records)

Nel backend i DTO sono implementati come **Java Records** (introdotti in Java 16): oggetti immutabili con campi finali, costruttore e getter generati automaticamente dal compilatore. Questo rende il codice conciso, sicuro e thread-safe.

5.3.1 HomeDto

Aggrega tutti i dati necessari alla home page: la lista dei tornei e la lista delle squadre. È il "container" principale restituito dall'endpoint **/api/home**.

```
public record HomeDto(  
    List<TorneoHomeDto> tornei,  
    List<SquadraHomeDto> squadre  
) {}
```

5.3.2 TorneoHomeDto

Rappresenta un torneo nella home. Contiene solo i dati visualizzati dalla card: id, nome, anno, descrizione, numero di squadre, flag preferito e le prossime partite.

```
public record TorneoHomeDto(  
    Long id,  
    String nome,  
    Integer anno,  
    String descrizione,  
    int numeroSquadre,  
    boolean preferito,  
    List<PartitaHomeDto> prossimePartite  
) {}
```

5.3.3 PartitaHomeDto

Rappresenta una partita riassuntiva per la home. Include gli ID delle squadre, i nomi, i goal, la data/ora formattata e lo stato. È un sottoinsieme molto più piccolo rispetto all'entità **Partita** (che ha luogo, arbitro, commenti, eliminata, etc.).

```
public record PartitaHomeDto(  
    Long id,  
    Long torneoId,  
    Long squadraCasaId,  
    Long squadraOspiteId,  
    String squadraCasa,  
    String squadraOspite,  
    Integer goalsHome,  
    Integer goalsAway,  
    String dataOra,  
    String stato  
) {}
```

5.3.4 SquadraHomeDto

Rappresenta una squadra nella home. Contiene solo id, nome, città e anno di fondazione, escludendo tutte le relazioni (giocatori, partite, tornei).

```
public record SquadraHomeDto(  
    Long id,  
    String nome,  
    String citta,  
    Integer annoFondazione  
) {}
```

5.3.5 RigaClassificaDto

DTO usato per rappresentare una riga della classifica di un torneo. A differenza degli altri, è una **classe** (non un record) perché contiene logica di business: il metodo **aggiungiPartita()** aggiorna punti, vittorie, pareggi, sconfitte e differenza reti.

```
public class RigaClassificaDto {  
    private Long squadraId;  
    private String nomeSquadra;  
    private int punti, partiteGiocate, vittorie, pareggi, sconfitte, golFatti, golSubiti;  
    private boolean ritirata;  
  
    public RigaClassificaDto(Long squadraId, String nomeSquadra) { ... }  
  
    public void aggiungiPartita(int golFattiPartita, int golSubitiPartita) {  
        this.partiteGiocate++;  
        this.golFatti += golFattiPartita;  
        this.golSubiti += golSubitiPartita;  
        if (golFattiPartita > golSubitiPartita) { this.vittorie++; this.punti += 3; }  
        else if (...) { ... }  
    }  
}
```

5.4 I DTO nel Frontend (TypeScript Interfaces)

Nel frontend, i DTO sono definiti come **interfacce TypeScript** nel file **frontend/src/types/index.ts**. Questo garantisce la **tipizzazione statica**: il compilatore controlla che i dati JSON ricevuti dal backend siano utilizzati correttamente, prevenendo errori di runtime.

Esempio: mappatura tra DTO Java e TypeScript

La corrispondenza è quasi uno-a-uno, ma con i tipi propri di TypeScript:

```
// Backend: Java Record
public record HomeDto(List<TorneoHomeDto> tornei, List<SquadraHomeDto> squadre) {}

// Frontend: TypeScript Interface
export interface HomeDto {
  tornei: TorneoHomeDto[];
  squadre: SquadraHomeDto[];
  n: number; // campo opzionale/legacy
}

// Backend: Java Record
public record TorneoHomeDto(Long id, String nome, Integer anno, ...)

// Frontend: TypeScript Interface
export interface TorneoHomeDto {
  id: number;
  nome: string;
  anno: number;
  descrizione: string;
  numeroSquadre: number;
  preferito: boolean;
  prossimePartite: PartitaHomeDto[];
}
```

5.5 Come vengono Prodotti nel Backend

Il **HomeService** (classe di servizio Spring) è responsabile della costruzione dei DTO. Legge le entità dal database, le trasforma in DTO e le restituisce al controller. Questo pattern è chiamato **Mapping Entity → DTO**.

```
@Service
public class HomeService {

    @Transactional(readOnly = true)
    public HomeDto getHomeData() {
        Utente utente = getUtenteLoggato();
        Set<Long> preferiti = (utente != null)
            ? preferitoService.getTorneoIdsPreferiti(utente)
            : Set.of();

        List<Torneo> entitaTornei = torneoService.findAllOrdinatiPerAnno();
        List<TorneoHomeDto> tornei = new ArrayList<>();
        for (Torneo t : entitaTornei) {
            tornei.add(toTorneoHomeDto(t, preferiti));
        }
        tornei.sort((t1, t2) -> Boolean.compare(t2.preferito(), t1.preferito()));

        List<Squadra> entitaSquadre = squadraService.findAll();
        List<SquadraHomeDto> squadre = new ArrayList<>();
        for (Squadra s : entitaSquadre) {
            squadre.add(toSquadraHomeDto(s));
        }
    }
}
```



```

    return new HomeDto(tornei, squadre);
}

private TorneoHomeDto toTorneoHomeDto(Torneo t, Set<Long> preferiti) {
    List<PartitaHomeDto> prossime = partitaService
        .getProssimePartite(t.getId(), N_PARTITE)
        .stream().map(this::toPartitaHomeDto).toList();
    boolean isPreferito = preferiti.contains(t.getId());
    return new TorneoHomeDto(
        t.getId(), t.getNome(), t.getAnno(), t.getDescrizione(),
        t.getSquadre().size(), isPreferito, prossime);
}
}

```

Notare che il servizio **non espone mai direttamente le entità JPA** (es. **Torneo**, **Squadra**): le trasforma esplicitamente in DTO prima di restituirle. Questo isola il dominio dal contratto API e permette di calcolare campi derivati (es. **preferito**, **numeroSquadre**) senza sporcare le entità.

5.6 Come vengono Utilizzati nel Frontend

I DTO TypeScript vengono usati in tre modi principali:

1. **Tipizzazione delle chiamate API:** il generic **api.get<HomeDto>** assicura che il tipo restituito sia corretto.
2. **Props dei componenti:** i componenti ricevono oggetti DTO tipizzati (es. **TorneoCardProps** con **t: TorneoHomeDto**).
3. **Stato locale:** lo stato React è tipizzato con i DTO (es. **useState<HomeDto | null>**).

Esempio di utilizzo in HomePage.tsx

```

import type { HomeDto } from '../types';

export default function HomePage() {
    const [data, setData] = useState<HomeDto | null>(null);

    useEffect(() => {
        getHome()
            .then((res) => setData(res.data))
            .catch((e) => setError(e.message));
    }, []);

    // data.tornei è di tipo TorneoHomeDto[]
    const torneiFiltrati = data?.tornei?.filter((t) => {
        const matchNome = t.nome.toLowerCase().includes(searchTerm.toLowerCase());
        return matchNome || ...;
    });
}

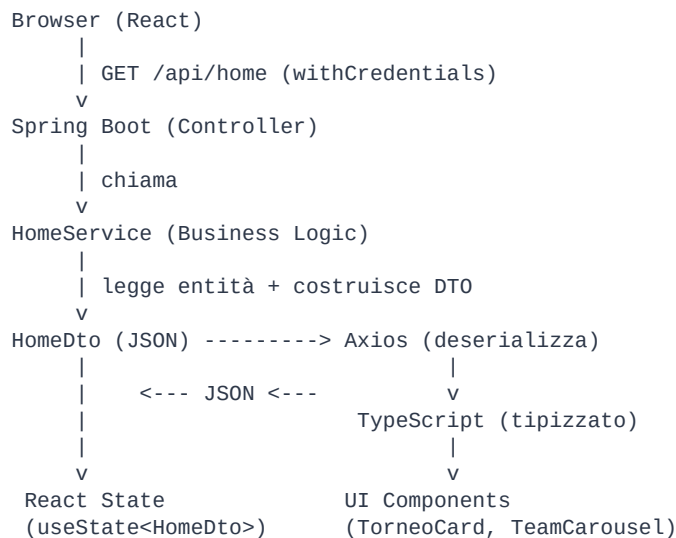
```

6. Flusso Dati Completo: Esempio HomePage

Per riassumere, ecco il percorso dei dati dalla richiesta alla visualizzazione:

1. **Utente** apre la pagina / nel browser.
2. **React** monta **HomePage**, che esegue **getHome()**.
3. **Axios** invia una richiesta GET a **/api/home** con il cookie JSESSIONID.
4. **Spring Boot** (HomeApiController) riceve la richiesta e chiama **HomeService.getHomeData()**.
5. **HomeService** legge le entità dal database, verifica i preferiti dell'utente loggato e costruisce i DTO.
6. **HomeApiController** serializza il **HomeDto** in JSON e lo invia come risposta HTTP.
7. **Axios** riceve il JSON e lo deserializza in un oggetto JavaScript.
8. **TypeScript** tratta i dati come **HomeDto** grazie al generic di Axios.
9. **HomePage** aggiorna lo stato con **setData(res.data)** e React re-renderizza i componenti figli.
10. **TorneoCard** e **TeamCarousel** ricevono i DTO come props e renderizzano l'UI.

Diagramma semplificato:



7. Sicurezza e Autenticazione

Il sistema gestisce l'autenticazione in modo ibrido: le pagine classiche (Thymeleaf) usano il form login di Spring Security, mentre il frontend React si integra con la stessa sessione tramite cookie. I punti chiave sono:

- **Sessione condivisa:** login e logout avvengono tramite endpoint standard di Spring Security; React si limita a verificare lo stato con **/api/auth/me**.
- **PrivateRoute:** componente che controlla **isAuthenticated** e reindirizza a **/login** se l'utente non è loggato.

- **Preferiti:** le operazioni sui preferiti sono protette; se l'utente non è autenticato, il backend restituisce **401 Unauthorized**.
- **CSRF:** le richieste di modifica (POST/DELETE) richiedono il token CSRF, ottenuto dall'endpoint `/api/csrf`.

8. Conclusione

Il frontend React del progetto **Tornei di Calcio** è un'applicazione moderna e ben strutturata che comunica con il backend Spring Boot tramite API REST JSON. L'uso dei **DTO** è fondamentale per garantire sicurezza, performance e manutenibilità: il backend trasforma le entità JPA in oggetti leggeri e immutabili (Java Records), mentre il frontend li riceve come interfacce TypeScript fortemente tipizzate. Questa separazione di responsabilità permette di evolvere il modello di dominio e l'interfaccia utente in modo indipendente, riducendo il rischio di errori e semplificando il testing su entrambi i lati.

Documento generato automaticamente dall'analisi del codice sorgente.